МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №7
Варіант 4

на тему «functools та operator»

Виконав:
студент групи ФеП-22
Корецький Андрій Вікторович

Перевірив:
аспірант
Остріков Данило Андрійович

Львів – 2026

Мета роботи. опанувати partial, reduce, singledispatch, total_ordering, itemgetter / attrgetter / methodcaller; побудувати конвеєр через часткове застосування та поліморфний API завантаження даних.

Теоретичні відомості

functools.partial фіксує частину аргументів функції й повертає нову функцію з меншим інтерфейсом — зручно для налаштовуваних етапів конвеєра. **functools.reduce** послідовно згортає колекцію; на ньому будується pipeline(value, *funcs).

Модуль **operator** містить оптимізовані замінники lambda: itemgetter (поля словника), attrgetter (атрибути об'єкта, кілька ключів для сортування), methodcaller (виклик методу).

@singledispatch створює генеричну функцію, реалізація якої обирається за типом першого аргументу. **@total_ordering** генерує <=, >, >= з __eq__ та __lt__; для чужих типів методи повертають NotImplemented.

Індивідуальне завдання (варіант 4)

Тема «Облік працівників». Дані: name, department, salary, experience, active. Відібрати активних, нормалізувати імена, обчислити премію та ранжувати. partial — для політики премії; singledispatch — для завантажувача; EmployeeOrder — порядок за salary / experience / name.

Хід роботи

1. Нормалізація через operator

itemgetter витягує поля сирого запису, methodcaller виконує strip / title / lower.

```
get_fields = itemgetter("name", "department", "salary", "experience", "active")

def normalize_name(raw: str) -> str:
    """'  iVAN   petrenko ' -> 'Ivan Petrenko'."""
    return " ".join(title(strip(raw)).split())
def normalize(record: Mapping[str, Any]) -> Employee:
    name, department, salary, experience, active = get_fields(record)
    return Employee(
        name=normalize_name(str(name)),
        department=lower(strip(str(department))),
        salary=float(salary),
        experience=int(experience),
        active=parse_bool(active),
    )
```

Лістинг 1 — фрагмент файлу core.py

2. Політики премії через partial і pipeline через reduce

Базова функція experience_bonus налаштовується через partial (standard_bonus). Конвеєр складається з partial-етапів: mapf, filterf, with_bonus, top_n.

```
def pipeline(value: Any, *funcs: Callable[[Any], Any]) -> Any:
    """pipeline(x, f, g, h) == h(g(f(x)))."""
    return reduce(lambda acc, f: f(acc), funcs, value)

def experience_bonus(
    employee: Employee, *, base_rate: float, per_year: float, cap: float
) -> float:
    """Премія = salary * min(base_rate + per_year * experience, cap)."""
    rate = min(base_rate + per_year * employee.experience, cap)
    return round(employee.salary * rate, 2)

standard_bonus = partial(experience_bonus, base_rate=0.05, per_year=0.01, cap=0.20)
def build_pipeline(
    *,
    bonus_policy: Callable[[Employee], float] = standard_bonus,
    n: int = 3,
    min_years: int = 0,
) -> Callable[[Iterable[Mapping[str, Any]]], list[Employee]]:
    """Збирає конвеєр із налаштованих (partial) етапів."""

    def run(records: Iterable[Mapping[str, Any]]) -> list[Employee]:
        result: list[Employee] = pipeline(
            records,
            mapf(normalize),
            filterf(is_active),
            filterf(partial(min_experience, min_years)),
            mapf(partial(with_bonus, bonus_policy)),
            rank,
            partial(top_n, n),
        )
        return result

    return run
```

Лістинг 2 — фрагмент файлу core.py

3. Поліморфне завантаження (@singledispatch)

load_employees приймає str (шлях або JSON-текст), pathlib.Path, відкритий текстовий файл і готову колекцію; підтримуються JSON і CSV.

```
@singledispatch
def load_employees(source: object) -> list[RawRecord]:
    """Генерична функція: реалізація обирається за типом ``source``."""
    raise TypeError(f"unsupported source type: {type(source).__name__}")
```

Лістинг 3 — фрагмент файлу loaders.py

4. @total_ordering — EmployeeOrder

Ключ порядку: salary спадно, experience спадно, name зростанням. a < b означає, що a стоїть у рейтингу вище, тому sorted() одразу дає рейтинг.

```
@total_ordering
class EmployeeOrder:
    """Порядок працівників у рейтингу: salary ↓, experience ↓, name ↑.

    ``a < b`` означає «a стоїть у рейтингу ВИЩЕ за b», тому
    ``sorted(map(EmployeeOrder, employees))`` одразу дає рейтинг.
```

```
    Реалізовано лише ``__eq__`` та ``__lt__``; ``<=``, ``>``, ``>=``
    генерує ``@total_ordering``.
    """

    __slots__ = ("employee",)
    def __init__(self, employee: Employee) -> None:
        self.employee = employee

    def _key(self) -> tuple[float, int, str]:
        e = self.employee
        return (-e.salary, -e.experience, e.name.casefold())
    def __eq__(self, other: object) -> bool:
        if not isinstance(other, EmployeeOrder):
            return NotImplemented
        return self._key() == other._key()
    def __lt__(self, other: object) -> bool:
        if not isinstance(other, EmployeeOrder):
            return NotImplemented
        return self._key() < other._key()

    def __hash__(self) -> int:
        return hash(self._key())

    def __repr__(self) -> str:
        e = self.employee
        return (
            f"EmployeeOrder({e.name!r}, salary={e.salary}, experience={e.experience})"
        )
```

Лістинг 4 — фрагмент файлу models.py

5. Тестування

22 тести покривають pipeline, partial, operator, усі джерела singledispatch, однаковий результат для JSON і CSV, reduce-агрегації та всі оператори порівняння.

```
def test_all_comparison_operators() -> None:
    hi = EmployeeOrder(emp("A", 900, 1))
    lo = EmployeeOrder(emp("B", 100, 1))
    same = EmployeeOrder(emp("A", 900, 1))
    assert hi < lo and hi <= lo and hi <= same
    assert lo > hi and lo >= hi and hi >= same
    assert hi == same and hi != lo
    assert not (hi > lo) and not (lo < hi)

def test_not_implemented_for_foreign_types() -> None:
    o = EmployeeOrder(emp("A", 1, 1))
    assert o.__eq__("x") is NotImplemented
    assert o.__lt__(5) is NotImplemented
    assert (o == "x") is False
    with pytest.raises(TypeError):
        _ = o < 5
    with pytest.raises(TypeError):
        _ = o >= "x"
```

Лістинг 5 — фрагмент файлу tests/test_lab07.py

Результати виконання

Результат запуску програми командою `python main.py`:

```
> python main.py
Завантажено 8 записів з sample_data.json

Top-3 (стандартна премія: 5% + 1%/рік, макс. 20%):
  1. Iryna Kravets      it     salary=   61000 exp= 4 bonus=   5490 total=   66490
  2. Andrii Shevchenko  it     salary=   52000 exp= 9 bonus=   7280 total=   59280
```

```
  3. Ivan Petrenko      it     salary=   52000 exp= 6 bonus=   5720 total=   57720

Top-3, стаж >= 3, премія за відділом (partial(department_bonus)):
  1. Iryna Kravets      it     salary=   61000 exp= 4 bonus=   9150 total=   70150
  2. Andrii Shevchenko  it     salary=   52000 exp= 9 bonus=   7800 total=   59800
  3. Ivan Petrenko      it     salary=   52000 exp= 6 bonus=   7800 total=   59800
Усі активні, сортування attrgetter('department', 'name'):
  1. Dmytro Savchuk     hr     salary=   33000 exp= 5 bonus=   9900 total=   42900
  2. Andrii Shevchenko  it     salary=   52000 exp= 9 bonus=      0 total=   52000
  3. Iryna Kravets      it     salary=   61000 exp= 4 bonus=      0 total=   61000
  4. Ivan Petrenko      it     salary=   52000 exp= 6 bonus=      0 total=   52000
  5. Sofia Lysenko      it     salary=   45000 exp= 2 bonus=      0 total=   45000
  6. Olena Koval        sales  salary=   38000 exp= 3 bonus=      0 total=   38000
  7. Taras Melnyk       sales  salary=   45000 exp= 2 bonus=      0 total=   45000

Фонд оплати (reduce): 357130.0
За відділами (reduce): {'it': 231640.0, 'sales': 89190.0, 'hr': 36300.0}
Інші джерела singledispatch:
  відкритий CSV-файл  : 8 записів
  str-шлях до CSV     : 8
  JSON-рядок          : [{'name': 'x'}]
  готовий list        : 8
```

Рисунок 1 — результат роботи програми (Lab_07)

Результат автоматизованого тестування `pytest -v` та статичних перевірок:

```
> black --check .
All done!
> ruff check .
All checks passed!
> mypy .
Success: no issues found
> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.3, pytest-9.2.0, pluggy-1.7.0 -- C:\labs\Функціональне
    програмування\.venv\Scripts\python.exe
rootdir: C:\labs\Функціональне програмування\functional-programming\Lab_07
configfile: pyproject.toml
testpaths: tests
plugins: platformdirs-4.14.5
collecting ... collected 22 items
tests/test_lab07.py::test_pipeline_applies_left_to_right PASSED          [  4%]
tests/test_lab07.py::test_partial_helpers PASSED                         [  9%]
tests/test_lab07.py::test_partial_bonus_policies PASSED                  [ 13%]
tests/test_lab07.py::test_with_bonus_returns_new_object PASSED           [ 18%]
tests/test_lab07.py::test_operator_tools PASSED                          [ 22%]
tests/test_lab07.py::test_normalization PASSED                           [ 27%]
tests/test_lab07.py::test_top_n_pipeline PASSED                          [ 31%]
tests/test_lab07.py::test_inactive_filtered_and_tie_by_name PASSED       [ 36%]
tests/test_lab07.py::test_min_years_and_n_edge_cases PASSED              [ 40%]
tests/test_lab07.py::test_input_records_not_mutated PASSED               [ 45%]
tests/test_lab07.py::test_reduce_aggregations PASSED                     [ 50%]
tests/test_lab07.py::test_predicates PASSED                              [ 54%]
tests/test_lab07.py::test_load_from_path_and_str PASSED                  [ 59%]
tests/test_lab07.py::test_load_from_open_file PASSED                     [ 63%]
tests/test_lab07.py::test_load_from_stringio_and_text PASSED             [ 68%]
tests/test_lab07.py::test_load_from_collections PASSED                   [ 72%]
tests/test_lab07.py::test_csv_and_json_give_same_result PASSED           [ 77%]
tests/test_lab07.py::test_unsupported_type_and_bad_json PASSED           [ 81%]
tests/test_lab07.py::test_all_comparison_operators PASSED                [ 86%]
tests/test_lab07.py::test_ordering_tie_breakers PASSED                   [ 90%]
tests/test_lab07.py::test_not_implemented_for_foreign_types PASSED       [ 95%]
tests/test_lab07.py::test_order_is_hashable PASSED                       [100%]
============================== 22 passed in 0.07s ==============================
```

Рисунок 2 — результати тестів і перевірок якості коду (Lab_07)

Висновок

Побудовано функціональний конвеєр обліку працівників на основі functools та operator. pipeline реалізовано через reduce, політики премії та етапи конвеєра налаштовуються через partial, поля та методи викликаються через itemgetter, attrgetter і methodcaller. Поліморфний завантажувач на singledispatch однаково обробляє рядки, Path, відкриті файли та колекції у форматах JSON і CSV. Клас EmployeeOrder з @total_ordering забезпечує багатокритеріальне ранжування. Усі вимоги перевірено тестами, mypy, black і ruff.